

Arrays

LEARNING OBJECTIVES

After studying this chapter, readers will be able to:

- Know about arrays
- Use array as an abstract data type
- Implement array using list
- Implement array using dictionary
- Implement array using class
- Define and use multidimensional arrays
- Apply array to manipulate polynomials
- Apply array to manipulate sparse matrices

In Chapter 1 we saw that the non-primitive data structure is divided into linear, non-linear and files. Again the linear data structures are divided into array, stacks and queues. In this chapter we will focus on the array linear data structures. Linear data structures can be implemented using arrays. The sequential organization of an array is a fixed size and uses continuous memory locations and, hence, the access time to any data element is the same, however large may be the array size. However, in-between insertions and deletions are expensive since they requires data shifting by several locations to keep the data consistent.

The elements are related to each other in a particular order or sequence. Some examples are:

1. even numbers less than or equal to 20
= {0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20}
2. days = {Sunday, Monday, ..., Saturday}
3. colors in a rainbow = {violet, indigo, blue, green, yellow, orange, red}

The operations one needs to perform in an order are to traverse from left to right and from right to left.

In Python there are several built-in data structures, such as a list, a tuple, and a dictionary, that can be used to implement arrays.

3.1 INTRODUCTION

An array, in almost all computer languages, is a built-in data type. In other words, an array can be declared just as an integer or a floating point number. The restriction on an array is that all the elements we wish to store in the array must be of the same data type and have a predetermined size.

An array, as a data structure, may be defined as a set of pairs (index, value) such that with each index, a value is associated:

- Index → indicates location
- Value → indicates actual value stored at the location

Thus, we can define an array as a set of finite, ordered collections of homogeneous elements that provides direct access to any of its elements in constant time. The data type of the array is the data type of its elements. We can access the elements by using appropriate subscripts in the array.

3.1.1 Array as an abstract data type

The following operations are provided with the abstract data type array (also see Table 3.1):

ADT Array Array_name

- Instance: create array_name(size)
- Pre-condition: At least one element required to perform any operation.
- Operations: Insert, Delete, Access

Table 3.1 Operation in Array datatype

Operation	Return
Declare create(n)	array.
Access(array, i)	value.
Insert(array, index, value)	array.
Delete(array, index)	array.
Access(create(), i)	error.
Access(store(array(index, i, x), j) = x if (i == j) else access(array, j)	value.

The `create()` function creates a new array.

`access(array, i)` accesses the i th element in the array.

The third operation, `insert(array, index, value)`, stores the given value at the index position of the array and returns the updated array.

The operation `delete(array, index)` deletes the value at a given index position of the array and returns the updated array.

The operation `access(create(), i)` gives an error because `create()` creates an empty array and we cannot access the i th element from a newly created array.

The last operation is easy to understand. The `access()` function returns the value in the i th position if $i == j$, else returns the value in the j th position.

Python arrays can be implemented using a list, a tuple or a dictionary and also by importing the `array` module or `numpy` module. A list is a mutable sequential and heterogeneous data structure. We can enforce the storing of homogeneous data by restricting the user to entering a specific type of data. As the size of the list is not static, it violates the notion of predetermined size. But we can enforce an array size.

A tuple can also be used for defining an array. A tuple is an immutable sequence of heterogeneous data structures. As it is immutable, we cannot perform the delete or insert operation once the tuple is defined. In order to perform this operation, we need to convert the tuple to a list, perform the operation and convert it back to a tuple.

A dictionary can also be used for defining an array. A dictionary is a mutable sequential and heterogeneous data structure that has key-value pairs as the data. As the key is unique, it can be treated as the index of an array and the value can be treated as data of the array. We can enforce the storing of homogeneous data by restricting the user to entering a specific type of data. As the size of the dictionary is not fixed, it violates the notion of predetermined size. While inserting and deleting elements, we need to adjust the key of the dictionary.

It is worth mentioning that while implementing an array using `list` or importing an `array` module or importing a `numpy` module, except for integer data, all other data are not kept in consecutive memory locations.

3.1.2 Implementation of array using list

Let us create an array of integers using a list and implement all the operations through Python. In this implementation, we have not enforced the fixed size of the list.

Python program 1

```
#create a list of integer
def create(n):
    lst=[]
    for i in range(n):
        no= int(input("Enter the number "))
        lst.append(no)
    return lst
```

The *create(n)* function creates a list with n elements. This method reads n integers appended to a list after the user enters all n integers and then the list is returned.

Program continues...

```
def display(a):
    print("The data in the array are :")
    print(a)
```

The *display(a)* displays all the elements of the array.

Program continues...

```
# Access a data in the list from a given location
def access(a,i):
    if 0 <= i <= len(a):
        print("The element at position ",i," is ",a[i] )
    else:
        print("The index is more than the size of array")
```

The *access(a, i)* method finds the element at i th location, if the location is not less than 0 or not greater than the size of the array.

Program continues...

```
# insert a data in the list at a given location
def insrt(a,i,x):
    if i < 0:
        print("Location is below the range")
        return a
    elif i > len(a) :
        print("Location is out of the range")
        return a
    else:
        a.insert(i,x)
        return a
```

The *insrt(a,i,x)* function adds an element at the *i*th location. This can be done by the *a.insert(i,x)* command while using a list.

Program continues...

```
# Delete a data from the list at a given location
def delit(a,i):
    if i < 0:
        print("Location is below the range")
        return a
    elif i > len(a) :
        print("Location is out of the range")
        return a
    else:
        del a[i]
        return a
```

Deletion of an element which is at *i*th location can be done by the *del a[i]* command.

Program continues...

```
# Main Program
n = int(input("Enter How many numbers in an array "))
Arr=create(n)
display(Arr)

print("\n--- Data at Location ---")
i = int(input("Element at which position element to get "))
access(Arr,i)

print("\n--- Insert at a Location ---")
j = int(input("Specify the Location to Insert "))
y = int(input("Specify the Data "))
Arr = insrt(Arr,j,y)
display(Arr)

print("\n--- Deletion at positon ---")
j = int(input("Specify the Location to Delete "))
Arr=delit(Arr,j)
display(Arr)
```

Output

```
Enter How many numbers in an array 5
Enter the number 10
Enter the number 20
Enter the number 30
Enter the number 40
Enter the number 50
[10, 20, 30, 40, 50]

--- Data at Location ---
Element at which position element to get 2
The element at position 2 is 30

--- Insert at a Location ---
Specify the Location to Insert 4
Specify the Data 60
[10, 20, 30, 40, 60, 50]

--- Deletion at position ---
Specify the Location to Delete 3
[10, 20, 30, 60, 50]
```

Some key facts to remember regarding Python lists:

- It is possible for the list to be homogeneous or heterogeneous.
- Python lists are one-dimensional by default. We can, however, make an n-dimensional list. However, it will still be a 1D list holding another 1D list.
- A list's elements don't have to be in order in memory.

3.1.3 Implementation of array using dictionary

Let us create an array of integers using a dictionary and implement all the operations. In this implementation we have not enforced the fixed size of the array.

Python program 2

```
def create():
    A={}
    n=int(input("\nEnter the number of elements :"))
    print("\n")
    for i in range(n):
        print("Enter data at ",i,end=": ")
        A[i]=int(input())
    return A
```

The *create()* function first creates an empty dictionary. And then for every key, a corresponding value is appended. This is done by the *A[i]=int(input())* command, which restricts the user to enter an integer value only.

Program continues...

```
def display(A):
    for i in range(len(A)):
        print("A[" + str(i) + "] = " + str(A[i]))
```

The *display(A)* function displays the elements of the dictionary in the form of an array.

Program continues...

```
# Access a data in the list from a given location
def access(A,i):
    if 0 <= i <= len(A):
        print("The element at position " + str(i) + " is " + str(A[i]) )
    else:
        print("The index is more than the size of array")
```

The *insrt(A, i, x)* method inserts an element with value *x* at *i*th location.

0	1	2	3	4
10	15	20	25	30

Figure 3.1 Sample Array

The insert method takes three arguments: the array itself, the position where the value is to be inserted and the value (Figure 3.1). If we want to insert a value at the *i*th position, we have to move all the elements of the array one position to the right starting from the *i*th position. Suppose *i* = 2 (Figure 3.2).

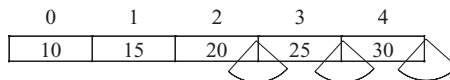


Figure 3.2 Movement of Data in Array-1

The size of the list will be increased by 1 after insertion of the element (Figure 3.3).

0	1	2	3	4	5
10	15		20	25	30

Figure 3.3 Movement of Data to a different position

We can now insert the element in the i th position. Let the element to be inserted be 17. The array now is as shown in Figure 3.4.

0	1	2	3	4	5
10	15	17	20	25	30

Figure 3.4 Insertion of Data at the i th location

Program continues...

```
def insrt(A,i,x):
    if i < 0:
        print("Location is below the range")
        return A
    elif i > len(A) :
        print("Location is out of the range")
        return A
    else:
        for j in range(len(A)-1,i-1,-1):
            A[j+1] = A[j]
        A[i]=x
        return A
```

The *delit*(A, i) method deletes the element at the i th location (Figure 3.5).

0	1	2	3	4	5
10	15	17	20	25	30

Figure 3.5 Original Array

Suppose we want to delete the element in the second position. Delete the data 17 at the second location (Figure 3.6).

0	1		3	4	5
10	15		20	25	30

Figure 3.6 Deletion of element from array

Observe that the value for key = 2 is deleted. Now we need to reorder the keys and values by saving the data of the location. The key in a dictionary is unique and it is not adjusted dynamically. We need to manage it by the program.

All the elements to the right of the second location (the element in the second position) are shifted one position to the left. This can be done by overwriting the second cell with the element in the third cell, and the element in the third cell is overwritten with the element in the fourth cell and so on. Also we need to reorder the keys of the dictionary (Figure 3.7).

0	1	2	3	4	5
10	15	20	25	30	30

Figure 3.7 Array after left movement of data

Lastly, delete the last cell (Figure 3.8).

0	1	2	3	4
10	15	20	25	30

Figure 3.8 Array after deleting the last element

Program continues...

```
# Delete a data from the list at a given location
def delit(A,i):
    if I < 0:
        print("Location is below the range")
        return A
    elif I >= len(A) :
        print("Location is out of the range")
        return A
    else:
        del A[i]

        for j in range(I,len(A)):
            A[j]=A[j+1]
        del (A[len(A)-1])
        return A
```

Program continues...

```
# main Program (Driver Function)
Arr = create()
print("\nThe element in the array is ")
display(Arr)

print("\n--- Data at Location ---")
i = int(input("Element at which position element to get "))
access(Arr,i)

print("\n--- Insert at a Location ---")
j = int(input("Specify the Location to Insert "))
y = int(input("Specify the Data "))
Arr = insrt(Arr,j,y)
display(Arr)

print("\n--- Deletion at position ---")
j = int(input("Specify the Location to Delete "))
Arr=delit(Arr,j)
display(Arr)
```

Output

```
Enter the number of elements :5

Enter data at 0: 1
Enter data at 1: 2
Enter data at 2: 3
Enter data at 3: 4
Enter data at 4: 5

The element in the array is
A[ 0 ] = 1
A[ 1 ] = 2
A[ 2 ] = 3
A[ 3 ] = 4
A[ 4 ] = 5

--- Data at Location ---
Element at which position element to get 3
The element at position 3 is 4

--- Insert at a Location ---
Specify the Location to Insert 4
Specify the Data 20
A[ 0 ] = 1
A[ 1 ] = 2
A[ 2 ] = 3
A[ 3 ] = 4
A[ 4 ] = 20
A[ 5 ] = 5

--- Deletion at position ---
Specify the Location to Delete 2
A[ 0 ] = 1
A[ 1 ] = 2
A[ 2 ] = 4
A[ 3 ] = 20
A[ 4 ] = 5
```

3.1.4 Implementation of array using import array

A one-dimensional array can be created by importing the *array* module.

Python program 3

```
from array import *
A = array('I', [1,2,3])
print(A)
```

To utilize the *array* module, we must first import it using the line *from array import **. Here *A* is the name of the array, and the method *array()* has two parameters. The type code, in this example, *i* is the first parameter, and the data is the second. Different type codes are given in Table 3.2.

Table 3.2 Data types and their size

Type code	C Type	Python Type	Minimum size in bytes
'b'	signed char	Int	1
'B'	unsigned char	Unicode character	1
'u'	Py_UNICODE	Int	2
'h'	signed short	Int	2
'H'	unsigned short	Int	2
'l'	signed int	Int	2
'L'	unsigned int	Int	2
'l'	signed long	Int	4
'L'	unsigned long	Int	4
'q'	signed long long	Int	8
'Q'	unsigned long long	Int	8
'f'	Float	Float	4
'd'	Double	Float	8

The program for accessing an array, inserting an element into the array and deleting an element from the array can be implemented as discussed in the list.

The Python array module provides the following advantages over Python lists:

- A list includes items of multiple data types, whereas the Python array module has elements of the same data type.
- To declare an array in Python, the array module must be explicitly imported, while in list, no module is required to be imported and no declaration is made.
- The Array module in Python may perform arithmetic operations.
- All items in the Python array module must be the same size.
- The term is most commonly used in the context of a prolonged succession of data items.
- Because addition, deletion and update operations are done on a single element at a time, modifying an array is complex.
- We'll need an explicit loop to print or access array items.
- In comparison to the list, it is a more compact in-memory size.

A list is the best solution if we need to store a reasonably brief sequence of elements and don't want to conduct any mathematical operations with it. Without importing any extra modules or packages, this data structure will let you store an ordered, mutable and indexed sequence of objects.

Consider utilizing an array if we have a long series of objects. This structure makes data storage more efficient. Use an array to conduct any numerical operations with your collection of objects. Arrays are widely used in data analytics and data science.

3.1.5 Implementation of array using NumPy

The module *Numpy* is a Python extension and is the most important module for scientific computing. *Numpy* arrays make it easier to do complex mathematical and other operations on enormous amounts of data. Such actions are often performed more quickly and with less code than utilizing Python's built-in methods. It performs quick and efficient operations on homogeneous data arrays.

The *numpy* package must be installed (if it is not already installed) before importing it. At the command prompt, type *pip install numpy* to install the package. It's a package for processing arrays in general.

Python program 4

```
import numpy as np
A = np.array([1,2,3],dtype = 'int')
print(A)
```

The package *numpy* must be imported before constructing an array. An alias (np) is used here to give a *numpy* shortcut. The function *np.array()* has two arguments to generate an array. The data (inside the square brackets) is the first argument, and the data type object (dtype) or type code of the array is the second argument. In this code, *dtype = 'int'* is used. This argument is optional for an array of integers, but it must be written for arrays of other data types.

In the example below, the type code is used as the second parameter. The second argument is optional for unsigned integers.

Python program 5

```
import numpy as np
A = np.array([1,2,3], 'I')
print(A)
```

The method for accessing array elements, inserting an element into the array and deleting an element from the array can be implemented as discussed in Section 3.1.2.

Some important points about *Numpy* arrays:

- *Numpy* arrays can also be used to construct an N-dimensional array in Python ().
- Arrays are homogeneous by default.
- It is possible to operate on individual elements.
- The *Numpy* array has a variety of functions, methods and variables that make calculating matrices easier.

Numpy arrays provide the following advantages over Python lists:

- *Numpy* arrays use less memory.
- When compared to the Python list, *Numpy* arrays are much faster.
- It is simple to use.
- It has many built-in optimized functions to perform array operations.
- Arrays are less versatile than lists. They have the ability to hold components of many data types, including strings. Furthermore, if you need to do mathematical computations on arrays and matrices, we should use *NumPy*.

We can implement an array using a list, Python *numpy* module or *array* module. Each method has its own benefits.

3.1.6 Implementation of array using class

We discussed class in Chapter 2. Now we will use class to implement an array. In this implementation, we have enforced the fixed size of the array.

Python program 6

```
# Creating an Array
class Arr:
    def __init__(self):
        self.maxSize = 10
        self.size = 0
        self.items = list()
```

The name of the array class is *Arr* and it can hold elements up to the *max-Size* (initialized to 10). The *size* is also initialized to 0. It also has an empty list. All the methods associated with the class are declared as public.

Let us define each method:

Program continues...

```
# Reading data for Array
def readArray(self):
    no = int(input("How many data to enter (Less than 10) "))
    if (no <= self.maxSize) :
        for I in range(no):
            data = input("Enter the data ")
            self.items.append(data)
            self.size += 1
    else:
        print("Number of Data is greater than Maximum size of array")
```

The *readArray ()* method reads character values into the list. It prompts how many characters to insert into the array. It checks if the input *array_size* is greater than the *maxSize* and if so, prints the appropriate message and comes out of the method. Otherwise, it prompts the user to insert data into the array.

Suppose the data inserted into the array is as shown in Figure 3.9.

0	1	2	3	4
A	B	C	D	E

Figure 3.9 Given array

Program continues...

```
# Inserting Data at Location
def addAtIndex(self,index,data):
    if ((index > self.size) or (self.size >= self.maxSize)):
        print("Addition of",data, "at index", index,"is not
        possible")
    else:
        self.items.append(None)
        for i in range(self.size, index, -1):
            self.items[i] = self.items[i - 1]
        self.items[index] = data
        self.size += 1
```

Insertions and deletions into or from an array are costly operations since they involve data movement to the right, and to the left in the case of deletions.

The *addAtIndex()* method takes two arguments, the position where the value is to be inserted (*index*) and the value (*data*). We need to check if the *index* at which we want to insert the data is beyond the current array size or the current array size is greater than or equal to the maximum size of the array. If either of the conditions is true, we cannot enter the data into the

array so an appropriate message is displayed and comes out of the method. Otherwise we can insert the data into the array.

If we want to insert a value at the j th position, we have to move all the elements of the array one position to the right starting from the j th position. Suppose $j = 2$ (Figure 3.10).

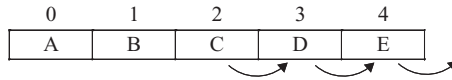


Figure 3.10 Movement of data in Array

When we insert an element into the array, the size of the array increases by 1. The elements are shifted one position to the right starting from the last element (the element at position $\text{size} - 1$) (Figure 3.11).

0	1	2	3	4	5
A	B	C	C	D	E

Figure 3.11 Array after data movement

We can now insert the element in the j th position. Let the element to be inserted be Z. The array now is as shown in Figure 3.12.

0	1	2	3	4	5
A	B	Z	C	D	E

Figure 3.12 After insertion of data

Program continues...

```
#Searching
def search(self, data):
    print("Searching for Element...",data)
    if data in self.items:
        position = 0
        for i in range(self.size):
            if(self.items[i] == data):
                break
            else:
                position += 1
        print('Element {} found at position {}'.format(data, position))
    else:
        print('This element is not in the Array!')
```

The method *search()* searches for the data passed as parameters. It is quite straightforward. We will keep searching from index 0 until the current size

of the array. If it matches, come out of the looping structure and print an appropriate message for finding the element. Otherwise print an appropriate message for not finding the data.

Deletion of a data requires searching for data in the array and if found then delete the data and reduce the size by 1, otherwise print an appropriate message.

Program continues...

```
#Delete a Data
def deleteData(self, data):
    print("Deletion ...",data)
    if data in self.items:
        self.items.remove(data)
        self.size -= 1
    else:
        print('This element is not in the Array!')
```

Unlike any other programming language, we need not write any code for searching for data. Searching of data is performed simply by the *if data in self.items* statement.

Suppose we want to delete the element in the second position. All the elements to the right of Z (the element in the second position) are shifted one position to the left. This overwrites the second cell with the element in the third cell, and the element in the third cell is overwritten with the element in the fourth cell and so on, and the blank cell moves to the extreme right.

Program continues...

```
# Delete an element at a location
def deleteAtLoc(self, loc):
    print("Deletion at location...",loc)
    if (loc == self.size):
        del self.items[loc]
        self.size -= 1
    else:
        print('This element is not in the Array!')
```

The *deleteAtLoc()* method takes one parameter, the location of data to be deleted. In Python deletion can be done by the *del* command.

Program continues...

```
# Main Program
myArray = Arr()
myArray.readArray()
print(myArray.items,"Array with" ,myArray.size,"Elements")
```

```

#Inserting data at location
if (myArray.size == 0):
    data = input("Enter the data ")
    myArray.addAtIndex(0,data)
    print(myArray.items,"Array with" ,myArray.size,"Elements")
else:
    print("\nInsert Data at location ")
    loc = int(input("Enter the location "))
    data = input("Enter the data ")
    myArray.addAtIndex(loc,data)
    print(myArray.items,"Array with" ,myArray.size,"Elements")

# Searching data
ele = input("\nEnter data to search ")
myArray.search(ele)

#Deleting Data
ele = input("\nEnter data to Delete ")
myArray.deleteData(ele)
print(myArray.items,"Array with" ,myArray.size,"Elements")

#Deleting data at a given location
loc = int(input("\nEnter location of data to Delete "))
myArray.deleteAtLoc(loc)
print(myArray.items,"Array with" ,myArray.size,"Elements")

```

In the main program, an object, *myArray* of the class *Arr*, is created. All the methods defined earlier are called to perform the operations.

Output

```

How many data to enter (Less than 10) 4
Enter the data A
Enter the data B
Enter the data C
Enter the data D
['A', 'B', 'C', 'D'] Array with 4 Elements

Insert Data at location
Enter the location 2
Enter the data Z
['A', 'B', 'Z', 'C', 'D'] Array with 5 Elements

Enter data to search B
Searching for Element... B
Element B found at position 1

Enter data to Delete C
Deletion ... C
['A', 'B', 'Z', 'D'] Array with 4 Elements

```

```
Enter location of data to Delete 2
Deletion at location... 2
['A', 'B', 'D'] Array with 3 Elements
```

3.2 MULTIDIMENSIONAL ARRAYS

A one-dimensional array can be used to store the values of a function of one variable such as $f(x) = 3x^4 + 8x^2 + 5x + 9$; the cells of the array taking values of f for different integer values of x . The values of f corresponding to the integer values of x from 0 to 5 can be stored in an array as shown in Figure 3.13.

0	1	2	3	4	5
9	5	8	0	3	0

Figure 3.13 Data values stored in one dimensional array

However, if we want to store the values of a function in two variables, say, x and y such as $f(x,y) = x^2 + 5xy + 3y^2$, a one-dimensional array is not very useful. We need a two-dimensional array, represented as a grid, the values in the grid corresponding to values of f for different values of x and y as shown in Figure 3.14.

Index y↓ x□	0	1	2	3	4	5
0	1	3	12	27	48	75
1	5	9	23	43	69	101
2	32	15	36	52	92	129

Figure 3.14 Data values stored in multi-dimensional array

We can declare such an array as `z[3][6]`, with 3 and 6 indicating the number of rows and columns, and the values can be accessed as

$f[0][0] = 1, f[0][1] = 3, \dots, f[2][5] = 129.$

Two-dimensional arrays are called matrices. There are two methods of representing matrices in the computer memory:

- 1. row-major method
- 2. column-major method

In the row-major method, all rows are stored one row after another starting from the first row like:

```
0 3 12 27 48 75 1 9 23 43 69 101 4 15 36 52 92 129
```

In the column-major method, all columns are stored from the first column like:

0 1 4 3 9 15 12 23 36 27 43 52 48 69 92 75 101 129

The programmer need not know which method was used to store the two-dimensional array.

The same method of storage can be extended to multidimensional arrays (arrays of more than 2 dimensions). A three-dimensional array, $A[m][n][p]$, is represented as $m \times n \times p$ dimensional matrices and these $n \times p$ matrices are stored one after the other starting from $m = 0$ in a row-major fashion. The element $A[i][j][k]$, therefore, occurs in the memory where the i th $n \times p$ matrix is stored. Therefore, we have to leave the first $(i-1)np$ locations starting from the first location where the three-dimensional matrix is stored, and search for $A[i][j][k]$ in the next $n \times p$ locations.

3.3 APPLICATIONS

Arrays have various applications and are widely used in programming and data structures. We will discuss two important applications, polynomial manipulation and sparse matrix implementation, in this section.

3.3.1 Polynomial manipulation

An array is a very useful data structure to implement polynomials in a computer. A polynomial in a single variable can be written as $P(x) = a_0 + a_1x + a_2x^2 + \dots + a_nx^n$. The degree of the polynomial is n and it contains $(n + 1)$ terms. A polynomial of n th degree may therefore be represented by the array $(a_0, a_1, a_2, \dots, a_n)$.

Where the i th element of the array represents the coefficient of x^i . All the elements of the array may not be non-zero. For example, the polynomial $3 - 2x - x^2 + x^4$ is represented by the array in Figure 3.15.

3	-2	-1	0	1
---	----	----	---	---

Figure 3.15 Polynomial in an Array

If we want to represent a ‘sparse’ polynomial $5 + x^{98}$ we need an array of size 99 in which most of the elements are zero (Figure 3.16).

5	0	0	0	0	0	-	-	-	98
---	---	---	---	---	---	---	---	---	----

Figure 3.16 Sparse polynomial in Array

This is called a sparse polynomial since there are very few non-zero elements and the terms with 0 coefficients occupy a lot of space and may not contribute much to the operations on the polynomial.

The sparsity of a polynomial is the ratio between the number of zeros and the total number of elements. In the polynomial $3 - 2x - x^2 + x^4$, the number of zeros is 1 (as the x^3 term is missing) and the total number of terms is 5. So the sparsity of the polynomial is $1/5$. Similarly, for the polynomial $5 + x^{98}$, the number of zero terms is 97 and the total number of terms is 99. So the sparsity is $97/99$.

We have discussed that every term in a polynomial is represented by a coefficient and an exponent. Python has many such constructs that are capable of representing a polynomial. We are using a dictionary to represent a polynomial.

A dictionary has two parts: a *key* and a *value*. In a dictionary, the key is unique so we will use it to represent the *exponent* and the value to represent the *coefficient* of a term in a polynomial.

We can represent the polynomial $3 - 2x - x^2 + x^4$ in a dictionary as {0:3, 1:-2, 2:-1, 3:0, 4:1}.

A polynomial is an example of ordered data structure. The abstract data type for a polynomial is:

ADT Polynomial

Instance: create a polynomial

Pre-condition: At least one term is required to perform any operation.

Operations: Addition of two polynomial

Subtraction of two polynomial,

Multiplication of two polynomial and

Evaluation of a polynomial

Let us define a polynomial as an object belonging to a dictionary.

Python program 7

reads a polynomial

```
def readPoly():
    p={}
    n=int(input("\nEnter Degree of Polynomial :"))
    print("\n")
    for i in range(n):
        print("Enter coefficient for x^",i,end=": ")
        p[i]=int(input())
    return p
```

The size of the polynomial is given in line 3. The for loop from lines 5 to 7 reads the coefficients and the corresponding exponents and creates the terms of the polynomial.

Program continues...

displays the polynomial in a readable form

```
def print_poly(p):
    res=""
    for expo,coeff in p.items():
        if expo == 0:
            res = str(coeff)

        else:
            if int(coeff) == 0:
                pass
            else:
                res = res + ' + ' + str(coeff)+"x^"+str(expo)
    print(res)
```

The print_poly(p) function displays on the screen the coefficients and exponents in the order of the terms of the polynomial.

Program continues...

adds two polynomials

```
def add_poly(p1,p2):
    p3={}
    i=0
    j=0

    # while both polynomial have elements
    while ( i < len(p1) and j < len(p2) ):

        if (i < j):
            p3[i]=p1[i]
            i+=1

        elif (i > j):
            p3[j]=p2[j]
            j+=1

        else:
            p3[i]= p1[i] + p2[j]
            i+=1
            j+=1

    # if first polynomial exhausted
    if (i >= len(p1)):
        while (j != len(p2)):
            p3[j]=p2[j]
            j+=1
```

```
# if second polynomial exhausted
if (j >= len(p2)):
    while (i != len(p1)):
        p3[i]=p1[i]
        i+=1
return p3
```

When we add two polynomials, the resultant polynomial is of the size of the larger of the two polynomials. Here p1 and p2 are two polynomials passed as parameters and defined in an empty dictionary, p3, to store the result. There are three cases we come across while adding two polynomials.

The first case is that while each polynomial has some terms, we will check each term of both polynomials. Starting with exponent 0, we will check if the exponent of a term in p1 is less than the exponent of the term in p2. If so, copy the coefficient of p1 to p3 at the exponent of p1. Otherwise, copy the coefficient of p2 to p3 at the exponent of p2. If the exponent of both terms are the same, add the coefficient of p1 and p2 to store it at the same location of p3.

The second case is that if there are some elements left out in polynomial p2 while there are no terms left in p1, copy the remaining terms of p2 to p3.

The third case is that if there are some elements left out in polynomial p1 while there are no terms left in p2, copy the remaining terms of p1 to p3. At last, the resultant polynomial, p3, is returned.

Program continues...

multiplies two polynomials

```
def mul_poly(p1,p2):
    p3={}
    #Initializing the Dictionary to 0
    for expol,coeff1 in p1.items():
        for expo2,coeff2 in p2.items():
            p3[expol+expo2]=0

    #Polynomial Multiplication
    for expol,coeff1 in p1.items():
        for expo2,coeff2 in p2.items():
            p3[expol+expo2]= p3[expol+expo2] + coeff1*coeff2

    return p3
```

Here we have taken an empty dictionary, p3, to store the product of two polynomials p1 and p2. The maximum terms possible in p3 is the sum of terms in p1 and terms in p2. Now assign the coefficient of each term of p3 to 0. The actual polynomial multiplication is done after that.

Program continues...**# The main Program**

```

poly1=readPoly()
print("\nThe Polynomial-1 is ")
print_poly(poly1)

poly2=readPoly()
print("\nThe Polynomial-2 is ")
print_poly(poly2)

print("\nThe Sum of Polynomials is ")
poly3=add_poly(poly1,poly2)
print_poly(poly3)

print("\nThe Product of Polynomials is ")
poly3=mul_poly(poly1,poly2)
print_poly(poly3)

```

Output

Enter Degree of Polynomial :3

Enter coefficient for x⁰: 1

Enter coefficient for x¹: 2

Enter coefficient for x²: -3

The Polynomial-1 is

1 + 2x¹ + -3x²

Enter Degree of Polynomial :6

Enter coefficient for x⁰: 0

Enter coefficient for x¹: 0

Enter coefficient for x²: 0

Enter coefficient for x³: 6

Enter coefficient for x⁴: 4

Enter coefficient for x⁵: 8

The Polynomial-2 is

0 + 6x³ + 4x⁴ + 8x⁵

The Sum of Polynomials is

1 + 2x¹ + -3x² + 6x³ + 4x⁴ + 8x⁵

The Product of Polynomials is

0 + 6x³ + 16x⁴ + -2x⁵ + 4x⁶ + -24x⁷

The same program can be written with very few codes using the *numpy* package.

Python program 8

```
import numpy as np

def display(p1):
    p={}
    # Convert the coefficient and exponents to a dictionary
    coeff = list(p1.c)
    coeff.reverse()
    n = len(coeff)

    for i in range(n):
        p[i]=coeff[i]

    # displaying in polynomial form
    res=""
    for expo,coeff in p.items():
        if expo == 0:
            res = str(coeff)
        else:
            if int(coeff) == 0:
                pass
            else:
                res = res + ' + ' + str(coeff)+"x^"+str(expo)

    print(res)

# Constructing polynomial
p1 = np.poly1d([1, 2, 5, 3])
print("Polynomials-1")
display(p1)

p2 = np.poly1d([4, 9, 3])
print("Polynomials-2")
display(p2)

# Addition of polynomials
add = np.polyadd(p1, p2)
print("Addition of two polynomials")
display(add)

# Subtraction of polynomials
sub = np.polysub(p1, p2)
print("Subtraction of two polynomials")
display(sub)

# Multiplication of polynomials
mul = np.polymul(p1, p2)
print("Multiplication of two polynomials")
display(mul)
```

```
# Roots of polynomial
print("The roots of polynomial-1 are")
root=p1.r
print(root)

# Solve polynomials at a value
print("The value of polynomial-1 at x=2 is ",p1(2))
print("The value of polynomial-2 at x=2 is ",p2(2))
```

We use built-in methods for adding and multiplying polynomials in the python *numpy* package. We have the `np.polyadd(p1, p2)` method for polynomial addition and the `np.polymul(p1, p2)` method for polynomial multiplication.

Output

```
Polynomials-1
3 + 5x^1 + 2x^2 + 1x^3
Polynomials-2
3 + 9x^1 + 4x^2
Addition of two polynomials
6 + 14x^1 + 6x^2 + 1x^3
Subtraction of two polynomials
0 + -4x^1 + -2x^2 + 1x^3
Multiplication of two polynomials
9 + 42x^1 + 63x^2 + 41x^3 + 17x^4 + 4x^5
The roots of polynomial-1 are
[-0.63136114+1.9158304j -0.63136114-1.9158304j -0.73727772+0.j      ]
The value of polynomial-1 at x=2 is 29
The value of polynomial-2 at x=2 is 37
```

3.3.2 Sparse matrices

A matrix is the most frequently used data structure to solve many engineering and scientific problems. As we had studied earlier, a matrix can be represented in a row-major or a column-major method in a computer. In engineering problems such as finite element analysis or in scientific problems in the analysis of nuclear reactions, the matrices can be very large and most of the elements may be zero. Such matrices are called sparse matrices. In either row-major method or column-major method, a huge amount of space is used to represent such matrices, even though there are very few non-zero elements that really contribute to the solution of the problem. We need an alternate representation of such matrices for optimum utilization of memory and minimum use of computer time. Consider the matrix in Figure 3.17.

2	0	0	0	0	0	8	9	0	0	0
3	5	0	0	0	0	3	0	0	0	0
0	0	0	0	1	9	7	0	0	0	0
1	3	4	0	0	0	0	0	0	0	6

Figure 3.17 Sparse matrix

It may be considered as a sparse matrix with 13 non-zero elements and 31 zeros. How many elements should be zero or what is the percentage of non-zero elements in the set of total elements to call a matrix sparse is not fixed. It depends on the programmer or the memory available. The sparse matrix shown in Figure 3.17 has 44 elements (integers including zeros) and hence, requires 44 memory locations in either row-major representation or a column-major representation. However, the 13 useful non-zero elements require a space of 13 memory locations only. Therefore, for the optimum utilization of space, we need an alternative representation.

Each element of a matrix is uniquely represented by its row number, its column number and its value. We might therefore store a matrix by a list of triples of the form (i, j, value). We might construct the list placing the three tuples in the increasing order of row numbers. These lists of tuples are placed one below the other in the increasing order of row numbers and with the same row number in the increasing order of column numbers is a matrix representation of the sparse matrix. This representation has $3 \times$ (number of non-zero elements) elements.

Thus, the sparse matrix in Figure 3.17 can be stored as shown in Figure 3.18.

	1	2	3
A[0]	0	0	2
A[1]	0	6	8
A[2]	0	7	9
A[3]	1	0	3
A[4]	1	1	5
A[5]	1	6	4
A[6]	2	4	1
A[7]	2	5	9
A[8]	2	6	7
A[9]	3	0	1
A[10]	3	1	3
A[11]	3	2	4
A[12]	3	10	6

Figure 3.18 Sparse matrix representation in an array

This contains 13 rows and 3 columns, requiring 39 memory space, 5 less than what is required for the original matrix. For large matrices this makes a considerable difference. In the matrix in Figure 3.18, we have 13 rows and 3 columns. The number of rows is equal to the number of non-zero

elements. From the first row onward, we represent information about non-zero elements. The first row says that the first row and the first column in the sparse matrix contain the non-zero element 2. Similarly the seventh row says that the first row and third column in the sparse matrix contains 1. Some of the operations we wish to perform on these matrices are ‘transpose’, ‘addition’ and ‘multiplication’.

Let us represent the sparse matrix as a two-dimensional array:

Python program 9

```
# function read a matrix
def readMatrix():
    rows = int(input("How many rows "))
    columns = int(input("How many columns "))
    # Initialize a Zero-matrix
    matrix = [ [ 0 for i in range(columns) ]
               for j in range(rows) ]
    # Reading data for the matrix
    for i in range(rows):
        for j in range(columns):
            print("Enter Matrix element",i,j, ":",end="")
            matrix[i][j] = int(input())
    return matrix
```

To create a sparse matrix we need to initialize a zero matrix with the required size. And then read the data for the sparse matrix.

Program continues...

```
# function display a matrix
def showMatrix(matrix):
    for column in matrix:
        for element in column:
            print(element, end =" ")
        print()
```

The function showMatrix(matrix) displays the elements of the matrix on the screen.

Program continues...

```
# function to convert the matrix into a sparse matrix
def ToSparseMatrix(matrix):
    sparseMatrix = []
    for i in range(len(matrix)):
        for j in range(len(matrix[0])):
            if matrix[i][j] != 0 :
                temp = []
```

```
        # appending row, column value and
        # element into the
        # sparse matrix
        temp.append(i)
        temp.append(j)
        temp.append(matrix[i][j])
        sparseMatrix.append(temp)

    return(sparseMatrix)
```

The function `ToSparseMatrix(matrix)` converts the matrix read earlier to a sparse matrix. We have taken an empty list, *sparseMatrix*, to store the sparse matrix. Whenever there is a non-zero value in the given matrix, append the row, column and the value to a *temp* list. And then append the *temp* list to the *sparseMatrix* list. So the *sparseMatrix* contains a list of lists.

The same program can be written with very few codes using *numpy* and *scipy* packages as the following program.

Python program 10

```
import numpy as np
from scipy.sparse import csr_matrix
# create a 2-D representation of the matrix
A = np.array([[0, 1, 0, 5],
              [0, 0, 2, 0],
              [0, 1, 0, 2]])

print("Dense matrix-1 :\n", A)
# convert to sparse matrix representation
S1 = csr_matrix(A)
print("Sparse matrix-1: \n",S1)
```

Output

```
Dense matrix-1 :
[[0 1 0 5]
 [0 0 2 0]
 [0 1 0 2]]
Sparse matrix-1:
(0, 1)      1
(0, 3)      5
(1, 2)      2
(2, 1)      1
(2, 3)      2
```

3.3.2.1 Transpose

As we know, the transpose of a matrix is the matrix obtained by inter-changing rows and columns. In other words, the element in the *i*th row and

j th column will be in the j th row and i th column in the transposed matrix. The transpose of the matrix in Figure 3.18 is shown in Figure 3.19.

	1	2	3
A[0]	0	0	2
A[1]	0	1	3
A[2]	0	3	1
A[3]	1	1	5
A[4]	1	3	3
A[5]	2	3	4
A[6]	4	2	1
A[7]	5	2	9
A[8]	6	0	8
A[9]	6	1	4
A[10]	6	2	7
A[11]	7	0	9
A[12]	10	3	6

Figure 3.19 Transpose of a matrix in sparse matrix

However, this does not correspond to the type of structure we had chosen (i.e., the increased row number and with the same row number increased column number) and it requires movement of rows and columns to bring it to the required form. To avoid this, we may proceed interchanging rows and columns starting from column 1 as follows:

for all elements in column j (j ranging from 0 to number of columns)

place ($i, j, value$) at ($j, i, value$)

Since the rows are originally in order, the transposed matrix will be in the correct order (increased row number and with the same row number increased column number).

Python program 9 continues...

```
def transposeMatrix(matrix):
    matrix=np.array(matrix)
    matrix[:, [0, 1]] = matrix[:, [1, 0]]

    for i in range(len(matrix)):
        for j in range(len(matrix)-i-1):
            if ( (matrix[j][0] > matrix[j+1]
                [0]) or (matrix[j][0] == matrix[j+1]
                [0] and matrix[j][1] > matrix[j+1][1])) :
                matrix[[j,j+1]] = matrix[[j+1,j]]

    showMatrix(matrix)
```

The above function transposes a sparse matrix. The statement `matrix[:,0, 1] = matrix[:,1, 0]` swaps the first column and second column of the sparse matrix. The two for loops arrange the rows in ascending order based on the elements of the first column. If two elements in the first column are the same, then it will put the rows in ascending order based on the values of the second column for those rows.

3.3.2.2 Sparse matrix addition

Let us write an algorithm to add two sparse matrices.

1. Let A and B be two matrices to be added and stored in C.
2. Let M and N be the number of rows in A and B, respectively. In other words, M – 1 and N – 1 are the number of non-zero elements in A and B, respectively.
3. The number of rows in C will be M + N – 1.
4. Two matrices can be added if and only if the number of rows and columns are the same.

`C[0][0] = A[0][0];`

`C[0][1] = A[0][1]`

`C[0][2] = M+N-2`

5. Let i, j and k be the indices into A, B and C, respectively, initialized to 1.

6. While `i <= M` and `j <= N`

if `(A[i][0] = B[j][0])` // if the row numbers are the same

if `(A[i][1] = B[j][1])` // column numbers are the same

{

`C[k,0] = A[i][0]`

`C[k,1] = A[i][1]`

`C[k][2] = A[i][2] + B[j][2]`

}

This says that “take the elements in the same row number and column number in matrices A and B, add them, and place it in the same row number and column number in C.”

7. Increment the indices: `i=i+1;j=j+1;k=k+1`

8. If `(A[i][0] = B[i][0])` and `A[j][1]<B[j][1]` copy the *i*th row of A into the *k*th row of C:

`C[k][0] = A[i][0];`

`C[k][1] = A[i][1];`

`C[k][2] = A[i][2];`

9. Increment `i=i+1;k=k+1`

10. Otherwise, i.e., `(A[i][0] = B[i][0])` and `A[j][1]>B[j][1]`)

11. `C[k][0] = B[j][0];`

`C[k][1] = B[j][1];` // copy the *j*th row of B into the *k*th row of C

`C[k][2] = B[j][2];`

```

12. Increment j=j+1;k=k+1;
}

```

In other words, add the elements if the row number and column numbers are the same; copy otherwise in the appropriate order.

Python program 9 continues...

```

# Addition of Sparse Matrix
def addSparse(m1,m2):
    resMatrix=[]
    l1=len(m1)
    l2=len(m2)
    i=0
    j=0

    while (i < len(m1) and j < len(m2)):
        temp=[]
        t1=[]
        t2=[]
        t1=m1[i]
        t2=m2[j]

        if ((t1[0] > t2[0]) or (t1[0] == t2[0])
            and (t1[1] > t2[1])):
            temp.append(t2[0])
            temp.append(t2[1])
            temp.append(t2[2])
            resMatrix.append(temp)
            j+=1
        elif((t1[0]< t2[0]) or (t1[0] == t2[0])
            and (t1[1] < t2[1])):
            temp.append(t1[0])
            temp.append(t1[1])
            temp.append(t1[2])
            resMatrix.append(temp)
            i+=1
        else:
            temp.append(t2[0])
            temp.append(t2[1])
            temp.append(t1[2] + t2[2])
            resMatrix.append(temp)
            i+=1
            j+=1
    if l1 < l2 :
        for i in range (j,l2):
            resMatrix.append(m2[i])

```

```
else:
    for i in range (i,ll):
        resMatrix.append(m1[i])

print("\nThe Addition of two Sparse Matrix is :")
showMatrix(resMatrix)
```

We have already discussed `readMatrix()`, `showMatrix(Matrix)`, `ToSparseMatrix(Matrix1)` and `transposeMatrix(matrix)`. Now we will discuss `addSparse(m1,m2)`. It takes two sparse matrices as input.

While there is some data in sparse matrix 1 and sparse matrix 2, take one row each time and perform the operations. Remember that each row of the sparse matrix contains a row, column and value of the original sparse matrix. The addition of sparse matrices has three cases.

The first case is when the row value of sparse matrix 1 is less than the row value of sparse matrix 2 or both of them are the same and the column value of sparse matrix 1 is greater than the column value of sparse matrix 2, then append the sparse matrix 2 to the resultant sparse matrix.

The second case is when the row value of sparse matrix 1 is greater than the row value of sparse matrix 2 or both of them are the same and the column value of sparse matrix 1 is less than the column value of sparse matrix 2, then append the sparse matrix 1 to the resultant sparse matrix.

Otherwise, append the row value of sparse matrix 2, column value of sparse matrix-2 and sum of the values of sparse matrix 1 and sparse matrix 2 to the resultant sparse matrix.

If the number of rows of sparse matrix 1 is exhausted, append the sparse matrix- 2 to the resultant sparse matrix. If the number of rows of sparse matrix 2 is exhausted, append the sparse matrix 1 to the resultant sparse matrix.

Python program 9 continues...

```
#Main Program
rows = int(input("How many rows  "))
columns = int(input("How many columns  "))
print("\nRead Matrix-1:  ")
Matrix1=readMatrix(rows,columns)
print("\nRead Matrix-2:  ")
Matrix2=readMatrix(rows,columns)

#convert to Sparse Matrix
sm1= ToSparseMatrix(Matrix1)
sm2= ToSparseMatrix(Matrix2)

print("\nThe Sparse Matrix-1 :  ")
showMatrix(sm1)
print("\nThe Sparse Matrix-2 :  ")
showMatrix(sm2)
```

```
addSparse(sm1,sm2)

#Transpose of Sparse Matrix
print("\nThe Transpose of Sparse Matrix-1 is")
transposeMatrix(sm1)
print("\nThe Transpose of Sparse Matrix-2 is")
transposeMatrix(sm2)
```

Output

```
How many rows 2
How many columns 3

Read Matrix-1:
Enter Matrix element 0 0 :0
Enter Matrix element 0 1 :0
Enter Matrix element 0 2 :7
Enter Matrix element 1 0 :0
Enter Matrix element 1 1 :6
Enter Matrix element 1 2 :7

Read Matrix-2:
Enter Matrix element 0 0 :0
Enter Matrix element 0 1 :0
Enter Matrix element 0 2 :1
Enter Matrix element 1 0 :2
Enter Matrix element 1 1 :0
Enter Matrix element 1 2 :0

The Sparse Matrix-1 :
0 2 7
1 1 6
1 2 7

The Sparse Matrix-2 :
0 2 1
1 0 2

The Addition of two Sparse Matrix is :
0 2 8
1 0 2
1 1 6
1 2 7

The Transpose of Sparse Matrix-1 is
1 1 6
2 0 7
2 1 7

The Transpose of Sparse Matrix-2 is
0 1 2
2 0 1
```

The same program can be written with very few codes using *numpy* and *scipy* packages as in the following program.

Python program 11

```
import numpy as np
from scipy.sparse import csr_matrix

# create a 2-D representation of the matrix
A = np.array([[0, 1, 0, 5],
              [0, 0, 2, 0],
              [0, 1, 0, 2]])

B = np.array([[0, 5, 0, 5],
              [0, 1, 2, 0],
              [0, 1, 0, 0]])

print("Dense matrix-1 :\n", A)
print("Dense matrix-2 :\n", B)

# convert to sparse matrix representation
S1 = csr_matrix(A)
S2 = csr_matrix(B)
S = S1 + S2
print("Sparse matrix-1: \n",S1)
print("Sparse matrix-2: \n",S2)
print("Addition of two Sparse matrix: \n",S)
```

Output

```
Dense matrix-1 :
[[0 1 0 5]
 [0 0 2 0]
 [0 1 0 2]]
Dense matrix-2 :
[[0 5 0 5]
 [0 1 2 0]
 [0 1 0 0]]
Sparse matrix-1:
(0, 1)      1
(0, 3)      5
(1, 2)      2
(2, 1)      1
(2, 3)      2
Sparse matrix-2:
(0, 1)      1
(0, 3)      5
(1, 2)      2
(2, 1)      1
(2, 3)      2
```

Addition of two Sparse matrix:

```
(0, 1)    2
(0, 3)    10
(1, 2)    4
(2, 1)    2
(2, 3)    4
```

Example: Matrix multiplication

```
m = int(input("How many rows in Matrix1? "))
n = int(input("How many columns in Matrix1? "))
Mat1 = [[0 for x in range(n)] for x in range (m)]
for i in range(m):
    for j in range(n):
        Mat1[i][j] = int(input("Enter the elements of
Matrix 1: "))

m1 = int(input("How many rows in Matrix2? "))
n1 = int(input("How many columns in Matrix2? "))

if (n != m1):
    print("Matrix multiplication NOT possible!!!")
    exit()

Mat2 = [[0 for x in range(n1)] for x in range (m1)]
for i in range(m1):
    for j in range(n1):
        Mat2[i][j] = int(input("Enter the elements of
Matrix 2: "))

print("\nMatrix 1 is ")
for i in range(m):
    for j in range(n):
        print(Mat1[i][j], end = " ")
    print()

print("\nMatrix 2 is ")
for i in range(m1):
    for j in range(n1):
        print(Mat2[i][j], end = " ")
    print()

Mat3 = [[0 for x in range(m)] for x in range (n1)]
for i in range(m):
    for j in range(n1):
        for k in range(m1):
            Mat3[i][j] += Mat1[i][k] * Mat2[k][j]

print("\nMatrix multiplication is ")
for i in range(m):
    for j in range(n1):
        print(Mat3[i][j], end = " ")
    print()
```

Output

```
How many rows in Matrix1? 2
How many columns in Matrix1? 3
Enter the elements of Matrix 1: 1
Enter the elements of Matrix 1: 2
Enter the elements of Matrix 1: 2
Enter the elements of Matrix 1: 3
Enter the elements of Matrix 1: 0
Enter the elements of Matrix 1: 4
How many rows in Matrix2? 3
How many columns in Matrix2? 2
Enter the elements of Matrix 2: 5
Enter the elements of Matrix 2: 0
Enter the elements of Matrix 2: 8
Enter the elements of Matrix 2: 1
Enter the elements of Matrix 2: 2
Enter the elements of Matrix 2: 3
```

Matrix 1 is

```
1 2 2
3 0 4
```

Matrix 2 is

```
5 0
8 1
2 3
```

Matrix multiplication is

```
25 8
23 12
```

Example: Row sum of a matrix

```
Mat = [[501, 453, 664],
        [502, 444, 646],
        [503, 222, 999]]
for i in range(len(Mat)):
    sum = 0
    for j in range(len(Mat[i])):
        sum = sum + Mat[i][j]
    print("The sum of ", Mat[i], "is : ", sum)
```

Output

The sum of [501, 453, 664] is : 1618

The sum of [502, 444, 646] is : 1592

The sum of [503, 222, 999] is : 1724

True–false questions

1. While using a list of integers, the memory location is consecutive.
a. True b. False
2. Numpy arrays use less memory.
a. True b. False
3. In a sparse matrix most of the elements are non-zero.
a. True b. False
4. Insertions and deletions into or from an array are costly operations
a. True b. False
5. All items in the Python Array module are not necessarily the same size.
a. True b. False

Fill-in-the-blank questions

1. An array is an ordered collection of _____ elements.
2. Numpy array can also be used to construct an _____ array in Python ().
3. Python lists are _____ by default.
4. A list is a mutable sequential and _____ data structure.
5. A list's elements don't have to be _____ in memory.

Multiple-choice questions

1. Find the output of the following code:

```
lst = [1,2,3,4,5]
print(lst[-1])
```

- a. 2
- b. 3
- c. 4
- d. 5

2. Find the output of the following code:

```
lst = [1,2,3,4,5]
print(list(lst[-3:-1]))
```

- a. [2, 3]
- b. [3, 4]
- c. [4, 5]
- d. [3, 5]

3. Find the output of the following code:

```
lst = [0,1,2,3,4,5]
lst.insert(0,1)
del lst[1]
print(lst)
```

- a. [0, 1, 2, 3, 4, 5]
- b. [1, 1, 2, 3, 4, 5]
- c. [2, 1, 2, 3, 4, 5]
- d. [3, 1, 2, 3, 4, 5]

4. Find the output of the following code:

```
lst = [0,1,2,3,4,5]
lst1 = lst
del lst1[1:2]
print(lst1)
```

- a. [0, 1, 3, 4, 5]
- b. [1, 2, 3, 4, 5]
- c. [0, 2, 3, 4, 5]
- d. [0,1, 3, 4, 5]

5. Find the output of the following code:

```
lst = [0,1,2,3,4,5]
lst1 = lst
del lst1[-1:-2]
print(lst1)
```

- a. lst and lst1 have the same length.
- b. It raises an error
- c. lst1 is a blank list.
- d. lst1 is longer than lst.

6. Find the output of the following code:

```
lst = [0,1,2,3,4,5]
lst1 = []
for i in lst:
    lst1.insert(0,i)
print(lst1)
```

- a. [0,1,2,3,4,5]
- b. [5,4,3,2,1,0]
- c. [0,0,0,0,0]
- d. None of the above

7. Find the output of the following code:

```
lst = [0,1,2,3,4,5]
for i in range(len(lst)):
    lst.insert(-1,lst[i])
print(lst)
```

- a. [1, 2, 3, 4, 5, 0, 0, 0, 0, 0, 0, 0]
- b. [1, 0, 0, 0, 0, 0, 0, 1, 2, 3, 4, 5]
- c. [0, 0, 0, 0, 0, 0, 0, 1, 2, 3, 4, 5]
- d. [0, 1, 2, 3, 4, 0, 1, 2, 3, 4, 0, 5]

8. Find the output of the following code:

```
lst = [[0,1,2,3,4,5] for i in range(3)]
print(lst[2][1])
```

- a. 0
- b. 1
- c. 2
- d. 3

9. Find the output of the following code:

```
my_list = [1,2,3,4,5,6]
count = 0
for item in my_list:
    if type(item) == str:
        continue
    count = count + 1
print(count)
```

- a. 2
- b. 4
- c. 6
- d. 0

10. Find the output of the following code:

```
m = 0
my_list_1= [1,2,5]
my_list_2= [1,3,2,6,5]
for x in my_list_1:
    for y in my_list_2:
        if x == y:
            m = m+1
print(m)
```

- a. 3
- b. 4
- c. 6
- d. 0

11. Find the output of the following code:

```
n1 = [10,20,30,40,50]
n2 = [10,20,30,40,50]
print(n1 is n2)
print(n1 == n2)
n1 = n2
print(n1 is n2)
print(n1 == n2)
```

- a. True,True,True,False
- b. True,False,True,True
- c. True,True,False,True
- d. False,True,True,True

12. Find the output of the following code:

```
prices = [30.5,'40.5',10.5]
total = 0
for price in prices:
    total += price
print(total)
```

- a. 81.5
- b. 30.1, 71.0, 81.5
- c. Expected indentation
- d. Raise an error ValueError

13. Find the output of the following code:

```
str1 = "I am learing Python"
str2 = str1.split('a')
print(str2)
```

- a. ['I ', 'm le', 'rning Python']
- b. ['Ia ', 'm lea', 'rning Python']
- c. ['I ', 'am le', 'arning Python']
- d. ['I ', 'am', 'learning', 'Python']

14. Find the output of the following code:

```
numbers = [0,1,2,3,4,5,6,7,8,9]
index = 0
while index < 10:
    print(numbers[index])
    if numbers(index) == 6:
        break
    else:
        index += 1
print(index)
```

- a. 0,1,2,3,4,5
- b. 1,2,3,4,5,6
- c. 0,1,2,3,4,5,6
- d. Raise an error

15. Find the output of the following code:

```
fruits = ['apple','orange','mango','banana']
for i in range(len(fruits)):
    fruits[i] = fruits[i][-1].upper()
print(fruits)
```

- a. ['B', 'A', 'N', 'A', 'N', 'A']
- b. ['A', 'O', 'M', 'B']
- c. ['E', 'E', 'O', 'A']
- d. ['L', 'G', 'G', 'N']

16. Find the output of the following code:

```
i = [10,20,[30,40],[50,60]]
count = 0
for i in range(len(l)):
    if type(l[i]) == list:
        count = count + 1
print(count)
```

- a. 1
- b. 2
- c. 3
- d. 4

17. Find the output of the following code:

```
x = [13,4,17,10]
w = x[1:]
u = x[1:]
y = x
u[0] = 50
y[1] = 40
print(x)
```

- a. [13, 40, 17, 10]
- b. [50, 40, 10]
- c. [13,4,17,10]
- d. [50,40,17,10]

18. Find the output of the following code:

```
a = ['a','b','c','d']
for i in a:
    a.append(i.upper())
print(a)
```

- a. ['a','b','c','d']
- b. ['A','B','C','D']
- c. Raise an error
- d. None of the above

19. Find the output of the following code:

```
numbers = [0,1,2,3,4,5,6,7,8,9]
```

```
i = '0'
```

```
index = 0
```

```
while(index < 10):
```

```
    if numbers[index] == 4:
```

```
        break
```

```
    else:
```

```
        index += 1
```

```
        i = i + str(index)
```

```
print(i)
```

- a. 1234
- b. 0123
- c. 12345
- d. 01234

20. Find the output of the following code:

```
my_list = [1,2,3,4,5,6]
```

```
count = 0
```

```
for item in my_list:
```

```
    if type(item != str):
```

```
        continue
```

```
    count += 1
```

```
print(count)
```

- a. 3
- b. 2
- c. 1
- d. 0

Descriptive questions

1. Consider the following matrix:
0 0 0 7 0 0 0 0
1 0 0 0 8 0 0 0
2 0 0 0 0 0 0 9
0 0 0 0 0 0 5 0
0 4 0 0 0 0 0 0
Represent this as a sparse matrix.
2. Create a list with the following elements and write a program to print it in the reverse order (without using the built-in function for reversing):
3 5 7 9 11 13 15
3. Write a program to delete the duplicate elements from a list.
4. How do you represent a polynomial in a computer? Write a program to add the following polynomials:
 $4x^4 + 2x^3 + 3x + 5$ $5x^5 + 3x^2 - 8$
5. Write a program to remove alternate elements from a list and create a new list.
6. An array A = {3, 5, 7, 9} is given and it is full. Write a program to insert a new element 11 to a list at a location without using the insert() function.
7. Given a list of integers, write a program to find the largest and smallest elements in the list without using max() and min().
8. Write a program to find a sub-list of a list from a given list.

Answers to true–false questions

1. True
2. True
3. False
4. False
5. False

Answers to fill-in-the-blank questions

1. homogeneous
2. N-dimensional
3. one-dimensional
4. heterogeneous
5. in order

Answers to multiple-choice questions

- | | | | | |
|-------|-------|-------|-------|-------|
| 1. d | 2. b | 3. b | 4. c | 5. c |
| 6. b | 7. d | 8. b | 9. c | 10. a |
| 11. d | 12. c | 13. a | 14. d | 15. c |
| 16. b | 17. a | 18. c | 19. d | 20. d |